

JAVA INTERVIEW PREPARATION

CHAPTER 7

COLLECTION CONTRACTS: CHOOSING THE RIGHT DATA STRUCTURE

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Java Collection Contracts: Choosing the Right Data Structure

Senior / Lead Java Developer Reading Chapter

The Java Collections Framework is not merely a catalogue of containers. Each interface communicates a semantic contract: whether order matters, whether duplicates are meaningful, whether elements are addressed by position or key, and how work should be consumed.

A senior developer should select a collection from the domain and access pattern rather than habit.

`ArrayList` and `HashMap` are excellent defaults in many situations, but the correct choice also depends on ownership, cardinality, mutation frequency, iteration guarantees, null handling, concurrency, memory layout, and overload behavior.

The first question is not "Which implementation is fastest?" It is "What promise should this API make about the data?"

The Main Collection Abstractions

The core interfaces describe different relationships.

```
Collection
|
+-- List      ordered sequence, duplicates allowed
|
+-- Set       uniqueness according to an equality or ordering rule
|
+-- Queue     elements waiting for processing
      |
      +-- Deque insertion and removal at both ends

Map           key-to-value association; not a Collection subtype
```

A `List<OrderLine>` says position and repeated values may matter. A `Set<Permission>` says duplicates have no meaning. A `Map<CustomerId, Customer>` says values are retrieved through unique keys. A `Queue<Job>` says elements are waiting for consumption according to an ordering policy.

Using the interface that matches the domain prevents invalid operations. A method that accepts `Collection<T>` does not promise indexed access. One that accepts `List<T>` does. A parameter of `Set<T>` communicates uniqueness even when the implementation happens to preserve order.

List: Ordered Positional Data

A List preserves encounter order and permits duplicates.

```
List<String> steps = new ArrayList<>();
steps.add("validate");
steps.add("authorize");
steps.add("authorize");
steps.add("capture");
```

The repeated authorization entry is retained because a List does not enforce uniqueness.

`ArrayList` is the general-purpose default. It stores references in a contiguous backing array, providing constant-time indexed access, efficient iteration, and amortized constant-time append. When capacity is exhausted, it allocates a larger array and copies references.

Insertion or removal near the front shifts later elements:

```
[A][B][C][D][ ]  
  
insert X at index 1  
  
[A][X][B][C][D]  
    ---> shifted references
```

[LinkedList](#) stores separate nodes connected by links. Adding at a known node can be constant time, but locating an index requires traversal. Node allocation, additional references, and poor CPU-cache locality make it a weak default for most application Lists.

If the dominant operation is queue or stack behavior, [ArrayDeque](#) is usually preferable to [LinkedList](#). If the dominant operation is random access and iteration, [ArrayList](#) is normally preferable.

Pre-size an [ArrayList](#) when a large expected size is known:

```
List<Order> batch = new ArrayList<>(expectedBatchSize);
```

Do not over-allocate without evidence. Retained unused capacity consumes memory, especially across many long-lived lists.

Set: Modeling Uniqueness

A Set prevents duplicates according to its equality model.

```
Set<CustomerNumber> customers = new HashSet<>();  
  
customers.add(new CustomerNumber("C-100"));  
customers.add(new CustomerNumber("C-100"));  
  
customers.size(); // 1 when value equality is correct
```

[HashSet](#) uses equality and hashing and is backed by a hash table. It provides average constant-time add, remove, and contains when hashes are stable and well distributed. It does not promise iteration order.

[LinkedHashSet](#) adds predictable encounter order, usually insertion order, with additional link overhead.

[TreeSet](#) stores elements in sorted order and defines duplicate positions through comparison. Operations are typically logarithmic.

```
Set<String> sorted = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);  
  
sorted.add("Java");  
sorted.add("JAVA");  
  
sorted.size(); // 1 according to the comparator
```

The comparator's zero result defines uniqueness for that [TreeSet](#). It may differ from [equals\(\)](#), which can surprise callers moving values between sorted and hash-based sets.

[EnumSet](#) is a compact, efficient representation for enum values and should normally be preferred over [HashSet](#) for one enum type.

```
EnumSet<Permission> allowed = EnumSet.of(  
    Permission.READ,  
    Permission.EXPORT  
);
```

An in-memory Set does not replace a database uniqueness constraint. Two application instances can independently accept the same value. Use the Set for local semantics and the database constraint for durable

cross-process correctness.

Map: Keyed Association

A Map associates one value with each key.

```
Map<CustomerId, Customer> customers = new HashMap<>();
customers.put(customer.id(), customer);

Customer found = customers.get(id);
```

Keys must have correct, stable equality and hashing for HashMap. Mutating an identity field after insertion can make the entry unreachable through ordinary lookup.

Map methods distinguish several common intentions:

```
map.put(key, value);           // replace or create
map.putIfAbsent(key, value);   // create only if absent
map.computeIfAbsent(key, this::load); // calculate on absence
map.merge(key, 1, Integer::sum); // combine values
```

The appropriate method expresses the complete state transition more clearly than separate `containsKey()` and `put()` calls. In a concurrent map, the atomic methods are essential because check-then-act sequences race.

`LinkedHashMap` provides predictable iteration order and can support access-order structures. `TreeMap` orders keys through natural ordering or a comparator. `EnumMap` is optimized for enum keys and preserves enum declaration order.

```
EnumMap<Status, Long> counts = new EnumMap<>(Status.class);
counts.merge(Status.APPROVED, 1L, Long::sum);
```

A Map is not a subtype of Collection because it stores entries with two roles. Its `keySet()`, `values()`, and `entrySet()` views expose collection perspectives backed by the Map.

Queue and Deque: Modeling Work Order

A Queue models elements waiting to be processed. The interface offers paired methods with different failure behavior.

Operation	exception form	special-value form
insert	<code>add(e)</code>	<code>offer(e)</code>
remove	<code>remove()</code>	<code>poll()</code>
examine	<code>element()</code>	<code>peek()</code>

`offer`, `poll`, and `peek` are often suitable when empty or full conditions are normal outcomes.

`ArrayDeque` is a resizable double-ended queue. It supports efficient insertion and removal at both ends and normally outperforms `LinkedList` for stack and queue behavior.

```
Deque<Task> tasks = new ArrayDeque<>();

tasks.addLast(normalTask);
tasks.addFirst(priorityTask);

Task next = tasks.pollFirst();
```

Use Deque instead of the legacy `Stack` class:

```
Deque<Frame> stack = new ArrayDeque<>();
stack.push(frame);
Frame current = stack.pop();
```

PriorityQueue orders elements by priority rather than insertion time. Iterating a **PriorityQueue** does not produce fully sorted order; repeated removal does.

```
Queue<Job> jobs = new PriorityQueue<>(
    Comparator.comparing(Job::priority).reversed()
);
```

Priority queues need a clear policy for equal priorities and starvation. A continuous stream of high-priority work can indefinitely delay lower-priority tasks.

Big-O Is a Starting Point, Not the Whole Cost

Complexity notation describes how work grows with input size. It does not capture constant factors, allocation, memory locality, resizing, hashing quality, or contention.

Structure	Typical strength	Important hidden cost
ArrayList	indexed access and iteration	shifting and resize copies
LinkedList	end operations at known nodes	nodes, traversal, poor locality
HashSet/Map	average key lookup	hashing, collisions, table memory
TreeSet/Map	sorted operations	comparisons and tree nodes
ArrayDeque	queue and stack operations	resizing and no indexed contract

An $O(n)$ scan over a contiguous array can outperform a theoretically better structure for small data because it uses cache-friendly memory and avoids hashing or node traversal.

Choose from realistic cardinality and operations. Benchmark only when the decision affects a hot path, and use representative keys, distributions, and mutation patterns.

Ordering Is a Contract

Different collections preserve different kinds of order:

- **ArrayList** preserves positional insertion order.
- **LinkedHashSet** and insertion-order **LinkedHashMap** preserve encounter order.
- **TreeSet** and **TreeMap** preserve comparator order.
- **PriorityQueue** guarantees only that the head has highest priority according to its comparator.
- **HashMap** and **HashSet** provide no stable iteration-order contract.

Code must not rely on an observed **HashMap** order. It may appear stable for one JDK, key set, and capacity and change after resize, deployment, or different input.

If an API promises order, choose and document the exact order. "Ordered" is incomplete without saying insertion, access, natural, comparator, timestamp, or business priority order.

Null Policies

Collection implementations differ in null support.

HashMap permits one null key and null values. **HashSet** permits null through its backing map. **ArrayList** permits null. **ConcurrentHashMap** rejects null keys and values. **ArrayDeque** rejects null because null is used by operations such as **poll()** to represent absence.

Allowing null often makes APIs ambiguous:

```
V value = map.get(key);
```

A null result can mean no mapping or a mapping to null. `containsKey()` distinguishes them, but a design that avoids null values is usually clearer.

Immutable factory methods such as `List.of`, `Set.of`, `Map.of`, and `copyOf` reject nulls. Define the null policy at the domain boundary rather than allowing implementation defaults to decide accidentally.

Views, Snapshots, and Ownership

Several collection methods return backed views.

```
Map<CustomerId, Customer> map = new HashMap<>();
Set<CustomerId> keys = map.keySet();

map.put(id, customer);
keys.contains(id); // true: the view reflects the map
```

Removing from the key-set view removes the Map entry. This is useful when intentional and dangerous when callers assume an independent Set.

`subList()` is also a view over part of a List. Structural modification outside the view can invalidate its behavior.

For public APIs, decide whether callers need:

- A live mutable view.
- A live unmodifiable view.
- An immutable snapshot.
- A stream or iterator.
- Domain operations instead of direct collection access.

```
public List<OrderLine> lines() {
    return List.copyOf(lines);
}
```

Returning a snapshot protects ownership but allocates. Storing an immutable snapshot internally may let the accessor return it directly. Avoid exposing internal mutable collections merely for convenience.

Iteration and Structural Modification

Many ordinary collection iterators are fail-fast. They detect some structural modifications and throw `ConcurrentModificationException` on a best-effort basis.

```
for (Order order : orders) {
    if (order.isCancelled()) {
        orders.remove(order); // commonly throws ConcurrentModificationException
    }
}
```

Use the iterator's removal operation or a collection method designed for the change:

```
orders.removeIf(Order::isCancelled);
```

Fail-fast behavior is a debugging aid, not thread safety. It is not guaranteed to detect every race, and code must not depend on the exception for correctness.

Concurrent collections provide different iteration contracts. `ConcurrentHashMap` iterators are weakly consistent: they do not throw `ConcurrentModificationException` and may reflect some changes during traversal, but they are not transactional snapshots.

`CopyOnWriteArrayList` iterators observe a stable array snapshot. Mutations copy the entire backing array, making it suitable for small read-mostly structures such as listener lists, not frequent writes.

Mutable, Unmodifiable, and Immutable Collections

A mutable collection supports structural changes.

An unmodifiable view rejects mutation through that reference but may reflect changes to its mutable backing collection.

```
List<String> source = new ArrayList<>(List.of("A"));
List<String> view = Collections.unmodifiableList(source);

source.add("B");
view.size(); // 2
```

An immutable snapshot does not change when the original changes:

```
List<String> snapshot = List.copyOf(source);
source.add("C");
snapshot.size(); // remains 2
```

These terms describe structural mutation. If elements themselves are mutable, changing an element can still alter observable state.

Document ownership. "Read-only" is not precise enough when a value may be a live view or a snapshot.

Concurrent Access and Compound Operations

Wrapping a collection with `Collections.synchronizedList` synchronizes individual methods, but compound operations still require coordination.

```
List<Job> jobs = Collections.synchronizedList(new ArrayList<>());

synchronized (jobs) {
    if (!jobs.contains(job)) {
        jobs.add(job);
    }
}
```

Iteration also requires synchronization on the documented mutex.

Concurrent collections provide operations designed for common atomic transitions:

```
cache.computeIfAbsent(key, this::loadValue);
counts.merge(key, 1L, Long::sum);
```

The choice between synchronized wrappers, concurrent collections, immutable snapshots, and thread confinement follows the entire operation, not whether one method happens to be synchronized.

Do not choose a concurrent collection as a substitute for defining higher-level invariants. Two maps updated separately do not become one atomic transaction because each is thread-safe.

Bounded Collections and Backpressure

Many production failures come from unbounded collection growth rather than incorrect element operations.

An executor queue, retry buffer, cache, deduplication Set, or in-memory batch can retain an unlimited object graph if no capacity or expiry policy exists.

```
BlockingQueue<Job> queue = new ArrayBlockingQueue<>(1_000);

if (!queue.offer(job, 50, TimeUnit.MILLISECONDS)) {
    throw new ServiceOverloadedException();
}
```

A bounded queue makes overload visible. The application can reject, slow producers, spill to durable storage, or shed lower-priority work.

An unbounded queue often delays the failure:

```
arrival rate > service rate
      |
      v
queue grows continuously
      |
      +-- waiting time increases
      +-- retained memory increases
      +-- timeouts create retries
      +-- eventual memory or latency failure
```

Capacity is part of the collection's production contract. Define maximum size, eviction or expiry, rejection behavior, and metrics.

Collection APIs Should Express Intent

Expose the weakest sufficient interface.

```
public List<Order> findRecentOrders(CustomerId id)
```

This promises ordered results and duplicates if meaningful. Returning `ArrayList<Order>` would unnecessarily commit callers to an implementation.

If the caller only iterates, `Collection`, `Iterable`, or `Stream` may be enough. If order and indexed access are contractually important, `List` is appropriate. If uniqueness matters, return `Set`.

Avoid accepting a collection only to mutate it unexpectedly:

```
void normalize(List<String> values) {
    values.replaceAll(this::normalizeValue);
}
```

The mutation should be explicit in the method name and documentation, or the method should return another collection:

```
List<String> normalized(Collection<String> values) {
    return values.stream()
        .map(this::normalizeValue)
        .toList();
}
```

Be aware that `Stream.toList()` returns an unmodifiable List in modern Java. Callers requiring mutation should create a mutable copy or use an appropriate collector.

A Practical Selection Process

Choose a collection by answering questions in order:

```
Need key-based lookup?
|
+-- yes --> Map
|
+-- no --> Need uniqueness?
|
+-- yes --> Set
|
+-- no --> Need queue/stack semantics?
|
+-- yes --> Queue or Deque
|
+-- no --> List or general Collection
```

Then refine the implementation:

- Does iteration order matter?
- Is sorted order required?
- What are the dominant reads and writes?
- What is the expected and maximum size?
- Are keys and elements immutable and correctly comparable?
- Is concurrent access required, and which compound operations must be atomic?
- Are nulls legal?
- Is the API returning a live view or snapshot?
- How is overload, eviction, or expiry handled?

This approach produces an explainable decision rather than a memorized implementation preference.

Production Perspective

Collection problems often appear as memory, latency, or correctness incidents. An unbounded queue causes request latency to climb for minutes before the heap fails. A service relies on HashMap iteration order and returns different results after a resize. A mutable key causes cache misses. A TreeSet drops values because its comparator returns zero. An unmodifiable view changes because another component owns the backing list. A synchronized wrapper protects single methods but not a compound invariant.

When diagnosing collection behavior:

- Identify the exact interface and implementation.
- Confirm equality, hashing, comparator, and null contracts.
- Measure cardinality, capacity, load, collision distribution, and retained memory.
- Determine whether the collection is bounded and who owns removal.
- Inspect whether the API returns a view, snapshot, or internal representation.
- Check iteration and concurrent-mutation guarantees.
- Correlate queue depth, waiting time, rejection, and downstream throughput.
- Use allocation and heap-retention evidence for memory questions.

Changing one implementation for another may move the symptom without fixing ownership or overload. Start from the semantic and operational contract.

Interview-Ready Summary

The Java Collections Framework expresses domain contracts. List models ordered positional data with duplicates, Set models uniqueness, Map models keyed association, and Queue or Deque models work ordering.

ArrayList is the usual List default because it provides efficient indexed access and cache-friendly iteration. LinkedList rarely wins in application code because traversal and node overhead dominate. ArrayDeque is normally preferred for queues and stacks.

Hash-based collections require stable equality and hashing. Sorted collections use comparison, which also defines duplicate positions. Linked variants preserve encounter order, while hash implementations do not promise iteration order. EnumSet and EnumMap are efficient choices for enum domains.

Big-O is only the starting point. Memory locality, allocation, resizing, collisions, comparisons, concurrency, and cardinality affect real performance. Public APIs should expose the weakest sufficient interface and explicitly define order, null policy, mutability, and snapshot-versus-view behavior.

Fail-fast iteration is not thread safety. Concurrent collections provide specific atomic operations and iteration semantics, but higher-level invariants still require design. Production collections must also be bounded or governed by expiry and eviction so overload becomes controlled behavior rather than hidden growth.

Revision Notes

- Choose a collection from the domain contract and dominant operations.
- List preserves order and duplicates.
- Set models uniqueness through equality or comparison.
- Map models key-to-value association and is not a Collection subtype.
- Queue and Deque model work ordering; ArrayDeque is the usual queue/stack default.
- ArrayList is the general-purpose List default for indexed access and iteration.
- LinkedList has node, traversal, and locality costs and is rarely the best default.
- HashSet and HashMap require stable, correctly implemented keys.
- TreeSet and TreeMap use comparator order; comparison zero defines duplicate positions.
- LinkedHashSet and LinkedHashMap provide predictable encounter order.
- EnumSet and EnumMap are preferred for enum domains.
- Big-O does not capture allocation, memory locality, resizing, collisions, or contention.
- Never depend on HashMap or HashSet iteration order.
- Define null policy explicitly; concurrent and immutable collections often reject null.
- Backed views reflect source mutation; snapshots do not.
- Fail-fast iterators are debugging aids, not concurrency controls.
- ConcurrentHashMap iteration is weakly consistent, not a transactional snapshot.
- CopyOnWriteArrayList suits small read-mostly collections with rare writes.
- Unmodifiable wrappers can still reflect backing-collection changes.
- Thread-safe methods do not automatically make compound operations atomic.
- Bound queues, caches, buffers, and deduplication structures.
- Expose the weakest sufficient interface and document order, ownership, and mutability.